

Constants Quick Reference

THE DECLARATION RECIPE

Every constant uses three keywords before the type and name:

```
public static final double kSpeedLimit = 0.8 ;
```

- **public** — any file in the project can read it
- **static** — belongs to the class itself; no new Constants() required
- **final** — value is locked at startup and can never be reassigned

Type	Meaning	BearBots example	Common uses in FRC
double	64-bit decimal	kSpeedLimit = 0.8	Speeds, distances, PID gains, voltages
int	Whole number	kLeftMotorPort = 0	Port numbers, CAN IDs, counts
String	Text	kProjectName = "BearBots"	Logger keys, auto mode names
boolean	True / false	kIsRightInverted = true	Inversion flags, feature toggles
long	64-bit integer	kLoopPeriodUs = 20_000L	Microsecond timestamps, large counts
float	32-bit decimal	kGearRatio = 6.75f	Lightweight calculations, sensor values

In the FRC wild you'll also see: *TunerConstants*, *FieldConstants*, *VisionConstants* — same public static final pattern, split into separate files as codebases grow.

■ Why not the other keywords?

private makes a constant unreadable from other files — defeats the purpose.

No static means you need an object instance to read it — no new Constants() needed.

No final means the value can be overwritten at runtime — it becomes a variable, not a constant.

CONSTANTS FILE PATTERN

Each class lives in its own file. Start small, add files as subsystems grow.

```
// DriveConstants.java
public final class DriveConstants{
    public static final double
        kSpeedLimit = 0.8;
    public static final double
        kDeadband = 0.1;
    public static final int
        kLeftMotorPort = 0;
    public static final int
        kRightMotorPort = 1;
}

// ArmConstants.java
public final class ArmConstants{
    public static final double
        kMaxAngleDeg = 120.0;
    public static final double
        kStowedAngleDeg = 5.0;
}

// AutoConstants.java
public final class AutoConstants{
    public static final double
        kDriveTimeSeconds = 2.0;
    public static final double
        kTurnTimeSeconds = 1.3;
}

// OperatorConstants.java
public final class OperatorConstants{
    public static final int
        kDriverControllerPort = 0;
}
```

HOW TO USE A CONSTANT

1 Import the constants file

```
// top of Drive.java
import frc.robot.DriveConstants;
```

2 Use a constant by name

```
double speed = DriveConstants.kSpeedLimit;
double dead = DriveConstants.kDeadband;
```

3 OR use the fully-qualified path (no import)

```
double speed =
    DriveConstants.kSpeedLimit;
```

4 Add a constant to an existing file

```
// inside DriveConstants.java
public static final double
    kMaxTurnSpeed = 0.6;
```

5 Create a new constants file

```
// new file: ScoopConstants.java
public final class ScoopConstants{
    public static final double
        kServoCarryPos = 0.3;
}
```

★ VS Code quick-fix (Ctrl+.)

Red underline on a class name? Hover it and press Ctrl+. → Add import. VS Code writes the line for you.

Constants Quick Reference

THE k NAMING RULE

Every WPILib constant: lowercase 'k' + UpperCamelCase. You'll see this across every WPILib source file.

Category	Example 1	Example 2	Example 3
Speeds / power	kSpeedLimit	kMaxTurnSpeed	kAutodriveSpeed
Angles	kMaxAngleDeg	kStowedAngleDeg	kScoringAngleDeg
Distances	kWheelCircumferenceMeters	kTrackWidthMeters	kWheelRadiusMeters
Timing	kDriveTimeSeconds	kTurnTimeSeconds	kWaitTimeSeconds
Ports / IDs	kLeftMotorPort	kRightMotorPort	kControllerPort
Strings	kProjectName	kAutoDefault	kLogPrefix
Booleans	kIsRightInverted	kEnableLogging	kIsSimulation

Rules:

- Always start with k
- Include unit when ambiguous
- No abbreviations
- Drop the class name
- Booleans: kIs or kEnable

Avoid

- speedLimit *x*
- kMaxAngle *x*
- kWC *x*
- DriveConstants.kDriveSpeed *x*
- kMotorInverted *x*

Use instead

- kSpeedLimit ✓
- kMaxAngleDeg ✓
- kWheelCircumferenceMeters ✓
- DriveConstants.kSpeed ✓
- kIsRightMotorInverted ✓

■ The unit rule prevents real bugs:

A function expecting radians and receiving degrees overshoots by 57×. Unit in the name = free sanity check every time you read the code.

COMMON MISTAKES

Magic number in subsystem file

```
double speed = 0.8; // x hardcoded
```

Hard to find; one wiring change means hunting across multiple files.

Missing static

```
public final double kSpeed = 0.8; // x
```

Requires a Constants() instance. Won't compile the way you expect.

Missing final

```
public static double kSpeed = 0.8; // x
```

Can be reassigned at runtime. Not a true constant.

No unit in name

```
kMaxAngle // x vs kMaxAngleDeg // ✓
```

Caller can't tell: degrees or radians? Wrong choice = robot overshoots by 57×.

Moved constant, forgot to update reference

```
speed = 0.8; // x still in Drive.java
```

Build fails. Good — the compiler tells you exactly where to fix it.

IN THE FRC WILD

BearBots starts with one file per subsystem. As codebases grow, teams add more. Some common ones you'll see in other FRC repos:

TunerConstants.java

Auto-generated by Phoenix Tuner X for TalonFX motors. Contains PID gains, gear ratios, encoder offsets. Never edit by hand.

FieldConstants.java

Field element positions: AprilTag poses, scoring zone coordinates. Used by vision and autonomous routines.

VisionConstants.java

Camera transforms, pipeline IDs, trust thresholds, and latency compensation values.

OperatorConstants.java

Controller port numbers and deadband values. Already in the BearBots project.

Constants.java

Some teams keep a single file with inner classes (DriveConstants, ArmConstants) instead of separate files. Either pattern works — pick one and stay consistent.

Pattern matters more than file count. Pick one and stay consistent.